

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Практическое занятие 9-10***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Практическая работа 8-9

Создание диаграммы классов

Цель:

Изучение программного инструмента объектно-ориентированного моделирования StarUML ™, использование функциональных возможностей для создания диаграммы классов.

Задачи:

- *Изучение нотаций и функциональных возможностей среды для создания диаграммы классов(*Выявление классов, стереотипы, документирование, пакеты)
- *Создание заданной диаграммы классов (выявление, назначение стереотипа, документирование, создание пакетов)*

Ход работы

Задание

Выполните визуальное моделирование систем с использованием UML на примере проектирования системы заказов интернет-магазина «Style».

Каждый товар в каталоге описывается артикулом, размерным рядом, ценой и фото с кратким описанием.

Покупатель может загрузить каталог товаров. Каталог не содержит разделы, имеет блочную структуру, состоит из набора товаров с фото, ценой и размерами. Покупатель складывает понравившиеся товары в корзину, при этом выбирая размер и количество необходимого товара данного артикула.

Корзину можно изменить: просмотреть, удалить товар, изменить количество позиций одного артикула, вернуться в каталог.

Когда покупатель делает заказ, он вводит свои личные данные, телефон и оплачивает его по банковской карте (если заказ не оплачен, то он и не сделан).

После того как сделан заказ, его можно забрать со склада через 1 рабочий день. Данные о заказе поступают сотруднику магазина, назовем его сотрудником отдела продаж, он проверяет наличие товаров и передает его кладовщику на комплектацию. Кладовщик, собрав заказ, делает отметку о готовности.

Заказ выдается со склада кладовщиком. Кладовщик выдает заказ и отмечает в системе, что заказ выдан.

Магазин не занимается доставкой заказов, не делает скидок. Для того чтобы ограничить масштаб задачи, мы не рассматриваем систему снабжения магазина новыми товарами. Этим занимается другая система, назовем ее Склад. Информация о проданных товарах (т.е. сделанных заказах) поступает также в систему Склад.

Поскольку на протяжении от создания до выдачи заказа, он проходит разные стадии, то будет разумно ввести понятие статуса заказа. Сотрудники магазина могут статус заказа изменять, а покупатель может проследить за сборкой заказа. В таком случае наша система предоставляет еще одну функцию: узнать статус заказа.

1. **Изучение нотаций и функциональных возможностей среды для создания диаграммы классов:**

Диаграммы классов

Диаграмма классов является частью логической модели системы и представляет статическую картину системы.

Для каждой системы строится не одна, а несколько диаграмм классов: возможно, что для каждого прецедента или сценария своя. На одних показывают подмножества классов, объединенные в пакеты, и отношения между ними, на других – отображают те же подмножества, но с атрибутами и операциями классов. Для представления системы разрабатывается столько диаграмм классов, сколько потребуется.

Основные элементы диаграмм классов

Дадим некоторые определения и опишем основные элементы нотации диаграмм классов.

Объект – это некоторая сущность реального мира или концептуальная (абстрактная) сущность.

Пример. Примерами объектов могут служить дом №4 по улице Садовая, сотрудник фирмы Иван Петров, ваш компьютер. Или нечто абстрактное: химическая формула, торговый заказ номер 456789, банковский счет клиента Петра Иванова.

Объект имеет четко определенные границы и значение для системы и характеризуется состоянием, поведением и индивидуальностью.

Состояние объекта – это одно из условий, в котором он может находиться. Состояние обычно изменяется со временем и характеризуется набором свойств, которые называются атрибутами.

Пример. Покупатель определяется его именем, адресом, телефоном, датой рождения.

Поведение определяет, как объект реагирует на запросы других объектов и что может делать сам объект. Поведение характеризуется операциями объекта.

Пример. Покупатель может добавить товар в корзину, просматривать каталог, удалять товар из корзины.

Индивидуальность означает, что каждый объект уникален, даже если его состояние идентично состоянию другого объекта.

Пример. Объекты Мария Петрова и Анна Седова уникальны, хотя каждый из них является покупателем магазина и имеет одинаковые поведение и состояния.

Как правило, в системе существует множество объектов имеющих одинаковое поведение, принимающих одинаковые состояния. Например, сотрудники фирмы, которых может быть несколько десятков, и данные о которых содержатся в базе данных, имеют одинаковые атрибуты – фамилию, имя, отчество, дату рождения, должность и др. – с разными значениями этих атрибутов, а также могут иметь схожее поведение – подать заявление на отпуск или перевод в другое подразделение. Для группировки объектов используются классы.

Класс – это описание группы объектов с общими свойствами (атрибутами), поведением (операциями), отношениями с другими объектами и семантикой.

Каждый класс является шаблоном для создания объекта. А каждый объект – это экземпляр класса. Важно помнить, что ***каждый объект может быть экземпляром только одного класса!***

Пример. Применительно к магазину «Style» можно сгруппировать сотрудников магазина, описав общий для них класс **Сотрудник**. Объект этого класса, например, Иван Петров, может включать в себя следующую информацию: имя, адрес, должность, размер заработной платы, кроме того этот объект может выйти в отпуск.

В нотации UML классы и объекты изображаются в виде прямоугольников (см. рис.1). Прямоугольник класса всегда делится на три секции (раздела), имя класса помещается в первую секцию, каждое слово в названии класса принято писать с большой буквы. Во второй и третьей секциях могут указываться атрибуты и операции класса соответственно, эти секции могут быть пустыми. Названия классов выбираются в соответствии с понятиями предметной области. Это должно быть существительное или словосочетание в единственном числе, наиболее точно характеризующее предмет. Класс должен описывать только одну сущность.

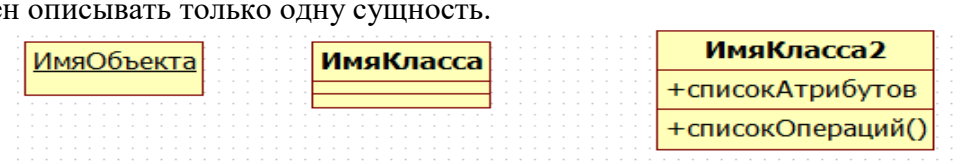


Рисунок 1. Изображение классов и объектов

Имя класса может быть простым, как это показано на рисунке 1, или составным (см. рис. 2). Составное имя класса состоит из самого имени класса и из имени пакета, которому принадлежит класс, разделенных двоеточием. Имя класса должно быть уникальным внутри пакета.

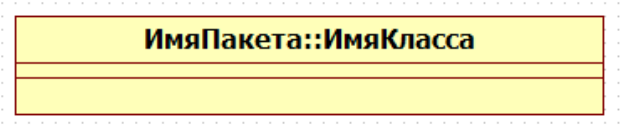


Рисунок 2. Составное имя класса

Составное имя объекта также состоит из имени объекта и имени класса, разделенных двоеточием. Объект может быть анонимным, если неизвестно его настоящее имя. Тогда на диаграмме объект изображается с именем, которое состоит из двоеточия и имени класса, которому принадлежит объект. Если пока неизвестен класс, экземпляром которого является объект, то изображается имя объекта после которого идет двоеточие. Такой объект называется «сиротой» (см. рис. 3).

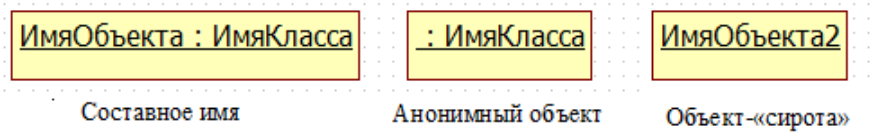


Рисунок 3. Именование объектов

Пример. Класс Сотрудник и объект этого класса - некоторого сотрудника - можно изобразить так, как показано на рисунке 4.



Рисунок 4. Класс и его объект

Мы дали объекту класса Сотрудник имя, совпадающее с именем класса.

Выявление классов

Выявление классов можно начать с изучения потока событий. Имена существительные в описании этого потока дадут понять, что может являться классом. В общем случае существительное может оказаться действующим лицом, классом, атрибутом класса или выражением, не являющимся ни действующим лицом, ни классом, ни атрибутом класса.

Если в ходе проектирования системы Вы уже построили диаграммы взаимодействия, перед тем, как приступать к построению диаграмм классов, то ищите на этих диаграммах похожие объекты. Например, у Вас может быть диаграмма последовательности, описывающая оформление заказа объектами Ивановым и Петровым. Обратите внимание на эти объекты: они имеют одинаковые свойства: имя, счет в банке и т.п. Значит, в системе должен появиться класс с именем Покупатель, который будет шаблоном объектов Иванов и Петров.

Некоторые возможные классы будут выявлены при рассмотрении трех стереотипов: **сущность (entity), граница (boundary) и управление (control).** Мы уже встречались со стереотипами отношений, когда говорили об отношениях на диаграммах прецедентов. Тот же принцип создания нового типа на основе уже существующего применим и для классов.

Стереотип – это механизм, позволяющий категоризировать классы. Он используется для создания нового типа элемента, в данном случае нового типа класса.

Например, Вы хотите выделить все экранные формы в модели. Для этого нужно создать стереотип Form (Форма).

Стереотипы помогают лучше понять ответственности каждого класса в модели, категоризировать выполняемые ими функции. В UML для этого применяют три основных стандартных вида стереотипов классов: классы-сущности, граничные классы и управляющие классы.

Класс-сущность содержит информацию, хранимую постоянно. Используется для моделирования данных и поведения с длинным жизненным циклом. Они могут представлять информацию о предметной области, а могут представлять элементы самой системы. Часто являясь абстракциями предметной области, они имеют наибольшее значение для пользователя, поэтому в их названиях применяются термины предметной

области. Если существует проект базы данных, то можно обратиться к изучению названий таблиц, многие из них станут классами-сущностями. Обозначаются классы-сущности стереотипом <<entity>> либо специальной пиктограммой (рис. 5).

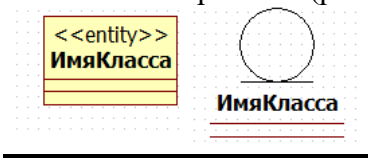


Рисунок 5. Обозначение классов-сущностей

Граничными классами называются классы, расположенные на границе системы со всем остальным миром, и т.о. они обеспечивают взаимодействие между окружающей средой и внутренними элементами системы.

Для вычисления пограничных классов необходимо исследовать диаграммы вариантов использования. Для каждого взаимодействия между актером и прецедентом нужно создать хотя бы один граничный класс. Обратите внимание, что если два действующих лица инициируют один прецедент, то они могут применять один общий пограничный класс для взаимодействия с системой. Обозначаются граничные классы именем стереотипа <<boundary>> либо специальной пиктограммой (рис. 6).

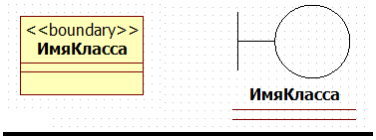


Рисунок 6. Обозначение граничных классов

Управляющий класс отвечает за координацию действий других классов. Они служат для моделирования последовательного поведения одного или нескольких прецедентов и координации событий, реализующих заложенное в них поведение. Обозначаются управляющие классы именем

стереотипа <<control>> либо специальной пиктограммой (рис. 7).

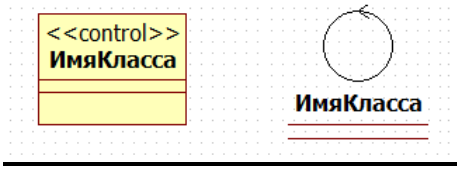


Рисунок 7. Обозначение управляющих классов

Управляющие классы можно представить, как «исполняющие» прецедент, поэтому у каждого варианта использования обычно имеется один управляющий класс, контролирующий последовательность событий этого прецедента. Они обычно зависят от приложения.

Управляющий класс делегирует ответственности другим классам. Сам он может получать мало сообщений, но отсылать множество. Его называют классом-менеджером. Он запускает альтернативные потоки и знает, как поступить в случае ошибки. На начальном этапе проектирования управляющие классы создаются для каждой пары актер/прецедент, в дальнейшем они могут объединяться, разделяться или исключаться.

Документирование классов

После того, как класс создан, информацию о нем необходимо документировать. Заметим, что документация предназначена для описания предназначения класса, а не его структуры.

Пример. Если в нашей модели присутствует класс Сотрудник, то хорошим описанием для него будет:

Сотрудник – это человек, работающий на фирме. Класс содержит информацию, необходимую для исполнения организацией своих обязанностей по отношению к сотруднику (начисление зарплаты, перевод на другую должность, увольнение и т.п.)

Плохим описанием будет описание структуры класса, которая может быть и так описана с помощью атрибутов. Например, плохое описание класса Сотрудник:

Имя, телефон, адрес, должность, зарплата.

В StarUML документирование классов выполняется также как документирование прецедентов. Нужно выделить класс, который вы хотите описать, открыть окно документирования Documentation на инспекторе модели и ввести описание класса.

Пакеты в языке UML

Если в модели немного классов, то легко ими управлять, однако многие системы содержат большое количество классов, поэтому необходим механизм, позволяющий классы группировать, что облегчает их повторное использование. Таким механизмом в UML являются пакеты.

Пакет (package) — общецелевой механизм для организации различных элементов модели в группы.

Подпакет (subpackage) — пакет, который является составной частью другого пакета.

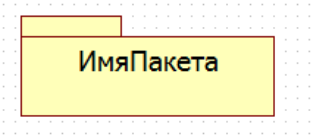
Пакет в логическом представлении модели – это объединение классов или других пакетов. С помощью объединения классов в пакеты мы можем получить представление о системе на более высоком уровне. Напротив, рассматривая пакет, мы получаем более детальное представление модели.

Объединять классы в пакеты можно как угодно, однако, существует несколько наиболее распространенных подходов.

- ✓ можно группировать классы по стереотипам: классы-сущности, граничные и управляющие классы.
- ✓ группировка классов по их функциональности: например, пакет классов, отвечающих за безопасность системы или пакет классов Работа с сотрудниками и т.п.
- ✓ наконец, применяют комбинацию двух указанных методов.

В дальнейшем можно вкладывать пакеты друг в друга.

Чаще всего пакет на диаграмме изображается в виде папки с закладкой с именем пакета



Главную диаграмму пакетов системы обычно помещают диаграмме Main представления **Logical View.** Для того чтобы создать пакет на диаграмме, нужно открыть рабочее поле диаграммы, щелкнуть по элементу пакет Package на панели элементов слева, затем щелкнуть по рабочему полю диаграммы в том месте, где вы хотите поместить пакет. В окне редактора свойств можно задать новое имя пакета.

Чтобы разместить классы по пакетам, используют метод перетаскивания: на навигаторе модели нужно перетащить, удерживая левую кнопку мыши, классы в соответствующие пакеты на навигаторе модели.

2. Построение диаграммы классов в StarUML

Диаграммы классов относятся к логическому представлению системы Logical View. На диаграмме Main представления Logical View обычно размещают главную диаграмму пакетов, а диаграммы классов помещают на другие листы этого представления. Для создания новой диаграммы классов выполним следующие шаги: щелкнуть правой кнопкой мыши по папке представления Logical View в навигаторе модели, в контекстном меню выбрать пункт Add Diagram, в списке выбрать диаграмму классов Class Diagram (рис. 8).

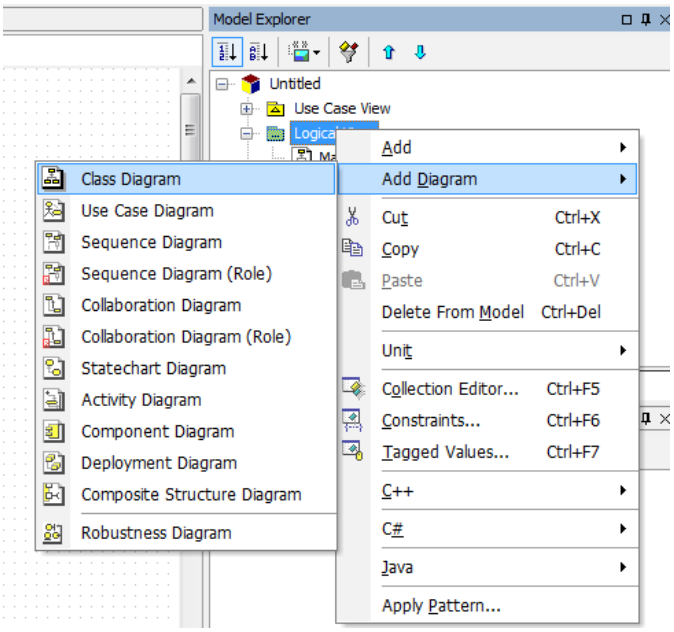


Рисунок 8. Добавление диаграммы классов

Будет создана новая диаграмма классов со стандартным именем ClassDiagram1, которое можно изменить в редакторе свойств диаграммы (рис. 9).

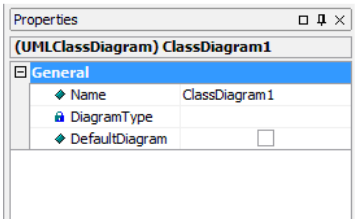


Рисунок 9. Редактор свойств диаграммы классов: изменение имени диаграммы

Рассмотрим сценарий **Оформление заказа**, который является внутренним потоком для прецедента **Заказ товара**. Опишем его классы.

Данный сценарий позволяет покупателю оформить заказ из корзины и оплатить его с использованием банковской карты.

*Покупатель, находясь в своей покупательской корзине, приняв решение о том, что он готов сделать заказ в магазине «Style», выбирает опцию «Оформить заказ». Как реагирует система на действия покупателя? Запускается сценарий Оформление заказа. Пользователь должен на специальной форме внести свои личные данные, подтвердить заказ или нет, и в зависимости от этого произвести оплату, затем получить подтверждение заказа. В системе появляется новый объект – **заказ покупателя**.*

Выбор граничных классов.

Таким образом, нужен будет хотя бы один **граничный класс**, который осуществляет связь между действующим лицом **Покупатель** и прецедентом **Оформить заказ.** Назовем его **ОформлениеЗаказа** (PlaceOrder). Этот класс знает, какие товары и в каком количестве были в корзине покупателя, их нужно перенести в заказ. Также этот класс может знать, пуста корзина покупателя или нет, и, если пуста, то вывести об этом соответствующее сообщение. Для того, чтобы сделать заказ, покупатель должен ввести свои личные данные, электронный адрес, телефон и данные кредитной карты. Для этих целей мы *введем еще один класс* **ВводЛичныхДанных** (EnterPersonalInformation). После того, как выбраны товары и введена личная информация покупателя, остается только проверить детали заказа и согласиться с ними или нет – для этого действия *введем класс* **ПроверкаДеталейЗаказа** (ConfirmOrder). Наконец, когда покупатель завершит оформление заказа, то на экране он видит номер заказа и подтверждение заказа отправляется покупателю на электронный адрес, для выполнения этих обязанностей *создадим еще один граничный класс* **ПодтверждениеЗаказа** (OrderConfirmation)

Выбор управляющих классов.

Создадим один управляющий класс, который будет распределять обязанности других классов и вызывать их операции при выполнении данного сценария - управляющий класс **МенеджерОформленияЗаказа** (PlaceOrderManager).

Выбор классов-сущностей.

В сценарии Оформление заказа речь идет о покупателе, заказе и товарах.

Создадим классы-сущности **Покупатель (Customer), Заказ (Order), Товар (Item).**

Возможно, что в ходе дальнейшего проектирования какие-то новые классы будут добавлены для этого сценария, а какие-то, напротив, из этой диаграммы удалены.

Задание 1

Постройте диаграмму классов сценария **Оформление заказа в StarUML**.

Создайте в **Logical View** диаграмму классов описанным выше способом. Переименуйте ClassDiagram1 в Place Order (Оформление Заказа).

Создайте новый класс **Покупатель (Customer**). Щелкните правой кнопкой мыши по Logical View в навигаторе модели, в контекстном меню выберите пункт Add (Добавить), затем выберите пункт Class (Класс). Новый класс будет создан и отобразится в навигаторе модели. На вкладке Properties (Свойства) измените имя класса на **Покупатель** (Customer).

Примечание. *При таком способе создания класса, StarUML создает новое пространство имен.* Если вы хотите создать класс, который носит такое же имя, как, например, действующее лицо на диаграмме прецедентов, то для того, чтобы StarUML «разрешил» использовать это имя еще раз, нужно создать класс способом, описанным выше. Теперь перетащите этот класс из навигатора модели на лист диаграммы Place Order (рис. 10).

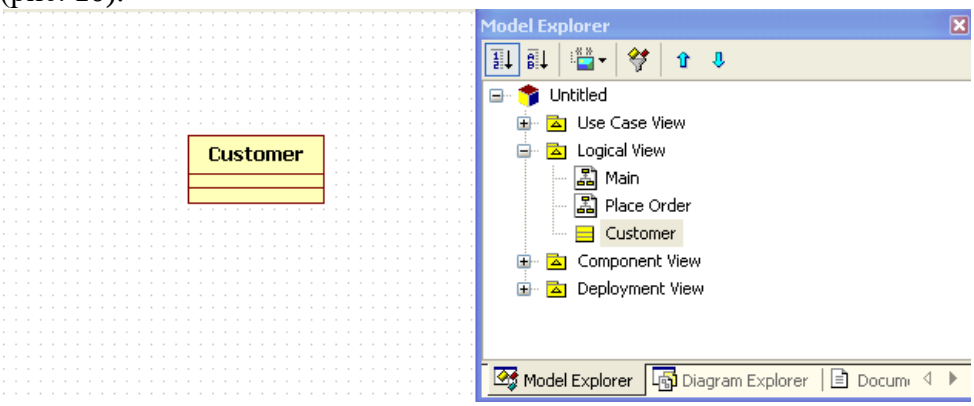


Рисунок 10. Создание класса Покупатель

Для того чтобы создать класс ОформлениеЗаказа (PlaceOrder), ***используйте другой метод.*** Слева на панели инструментов (Toolbox) щелкните изображение класса (Class), затем щелкните на листе диаграммы Place Order в том месте, куда необходимо поместить класс, при этом стандартное имя класса станет активным, давая возможность его изменить. Измените имя класса на ОформлениеЗаказа (PlaceOrder) (рис. 11).

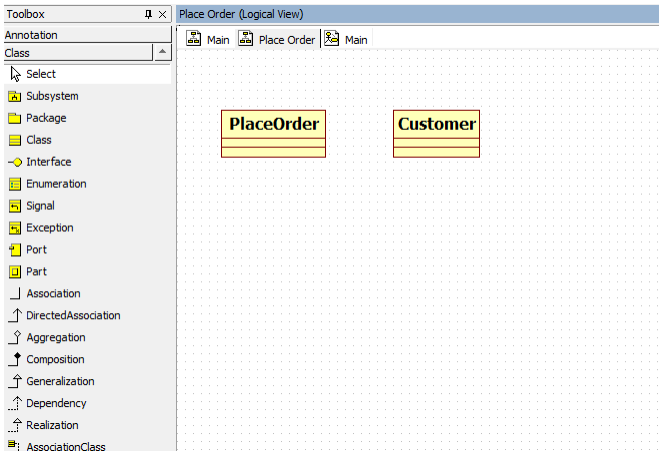


Рисунок 11. Создание класса ОформлениеЗаказа

Создайте все остальные классы.

Диаграмма классов представлена на рисунке 12.

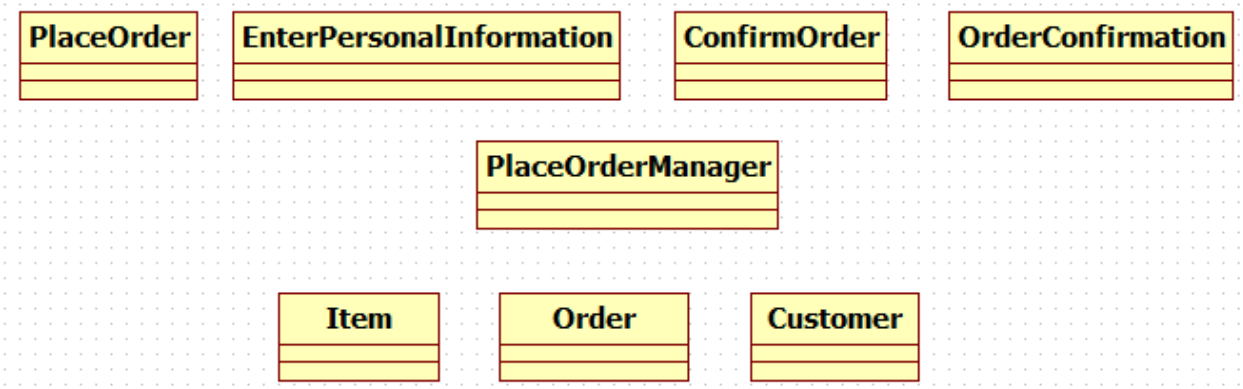


Рисунок 12. Диаграмма классов сценария Оформление заказа

Замечание. Создание диаграммы классов и взаимодействия выполняются на английском языке, так как эти диаграммы участвуют в генерации программного кода системы.

Назначение стереотипов

Чтобы присвоить классу один из стереотипов UML, нужно выделить класс щелчком правой кнопки мыши, открыть редактор свойств Properties на 15 инспекторе модели и выбрать раздел стереотип Stereotype (рис. 13).

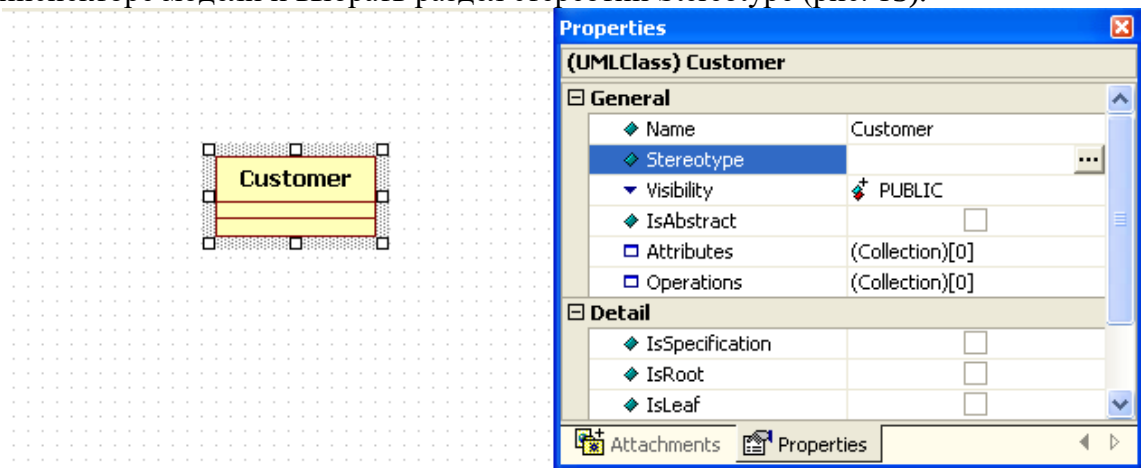


Рисунок 13. Назначение стереотипа классу

Если нажать на значок  , то в появившемся диалоге будет представлен список доступных стереотипов с кратким описанием и графическим обозначением. Для того чтобы присвоить классу стереотип, нужно выбрать его в списке, выделить и нажать кнопку OK (рис. 14).

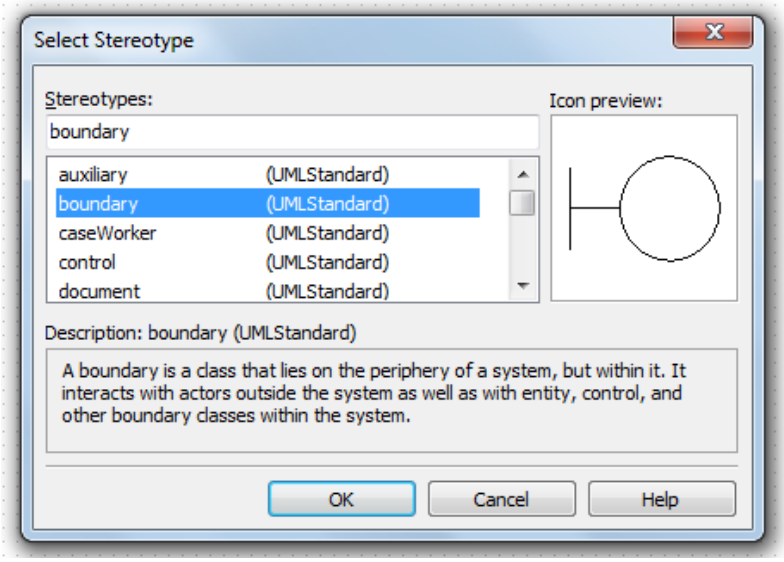


Рисунок 14. Выбор стереотипа

После присвоения классу стереотипа его внешний вид изменится. Рядом с именем класса появится имя стереотипа, заключенное в угловые скобки (рис. 15).

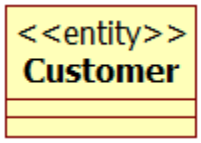


Рисунок 15.Отображение стереотипа класса 16

Задание 2. Присвойте классам сценария **Оформление заказа** соответствующие стереотипы. Диаграмма классов изменится (см. рис. 16).

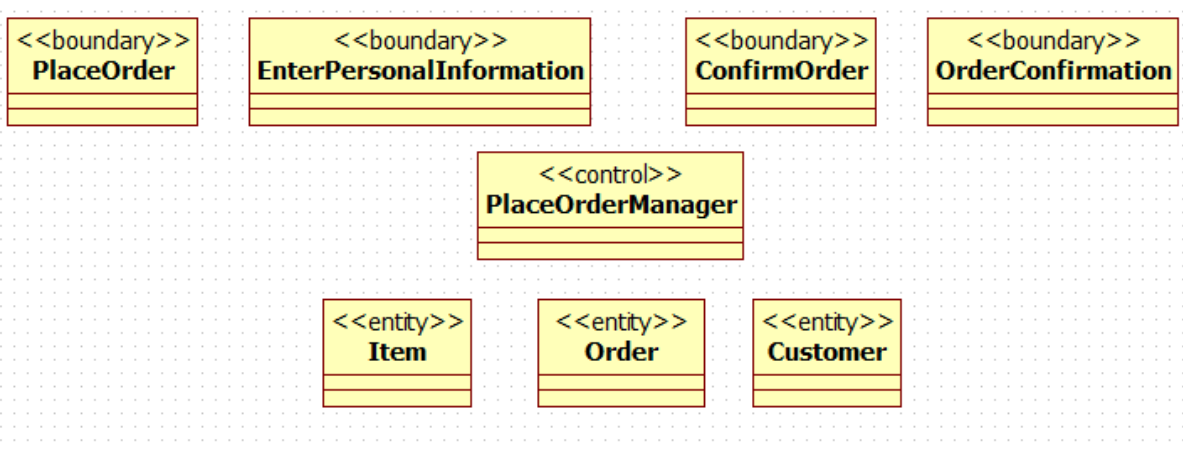


Рисунок 6. Диаграмма классов сценария Оформление заказа со стереотипами

Можно отобразить стереотипы классов с помощью пиктограмм. Для этого нужно выделить класс, щелкнуть по выделенной области правой кнопкой мыши, в контекстном меню выберите пункт Format, затем выберите пункт Stereotype Display, далее в списке выберите Iconic (рис.17).

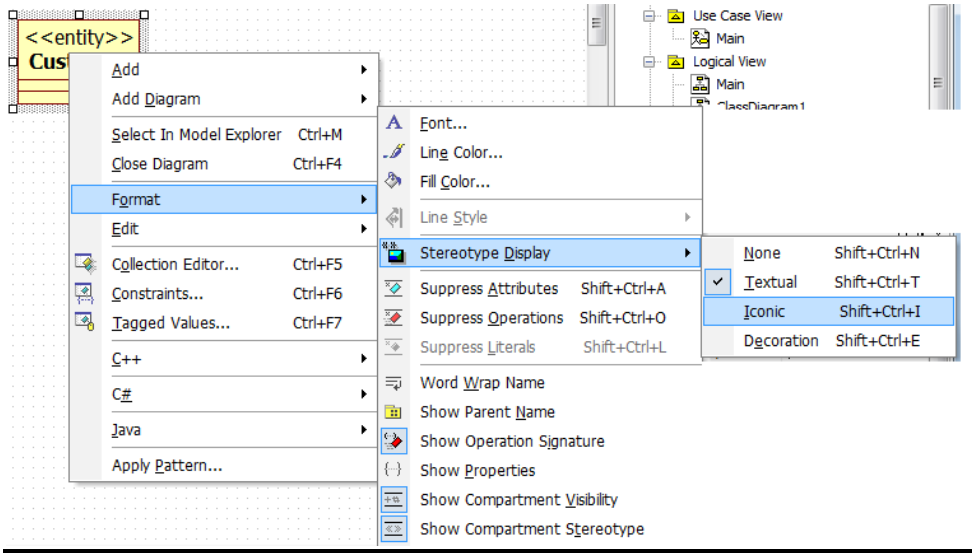


Рисунок 17. Задание отображения стереотипов классов в виде пиктограмм

Классы будут отображаться как пиктограммы (рис. 18).

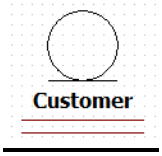


Рисунок 18. Отображение классов с помощью пиктограмм стереотипов

Задание 3. Диаграмму классов сценария Оформление заказа отобразите с помощью пиктограмм (см. рис. 19).

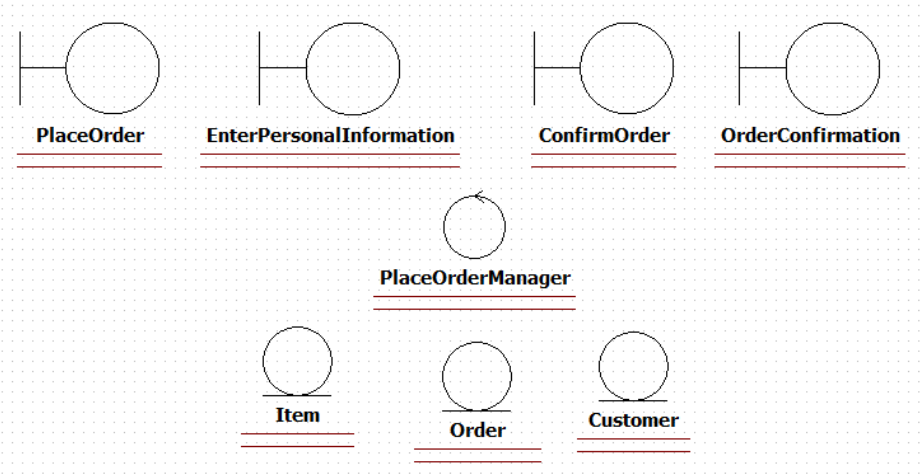


Рисунок 19. Диаграмма классов сценария Оформление заказа с помощью пиктограмм стереотипов

Задание 3 Созданные ранее классы сценария **Оформление заказа** сгруппируйте по пакетам, диаграмму пакетов поместите на листе Main представления Logical View.

Создайте пакеты Граничные классы, Классы-сущности, Управляющие классы (рис. 20).

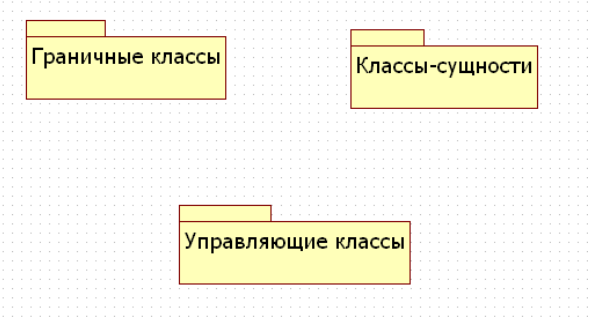


Рисунок 20. Диаграмма пакетов

Перетащите на навигаторе модели класс Customer (Покупатель) в пакет Классы-сущности (рис. 21).

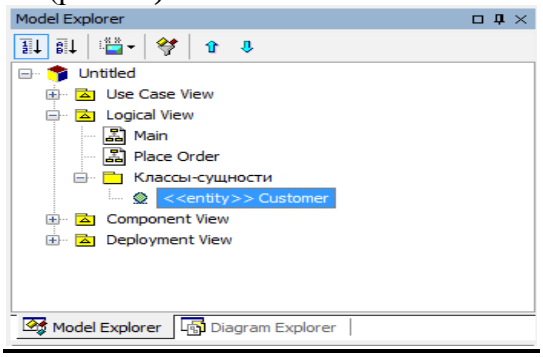


Рисунок 21. Перемещение классов в пакеты

Аналогично классы Item (Товар) и Order (Заказ) разместите в пакете Классы-сущности, классы ОформлениеЗаказа (PlaceOrder), ВводЛичныхДанных (EnterPersonalInformation), ПроверкаДеталейЗаказа (ConfirmOrder) и ПодтверждениеЗаказа (OrderConfirmation) разместим в пакете Граничные классы, а класс МенеджерОформленияЗаказа (PlaceOrderManager) – в пакете Управляющие классы.

***Примечание**. Внешний вид диаграмм классов Place Order и диаграммы пакетов Main не изменится.*

3. <u>Оформить отчет по плану:</u>

1. *Наименование работы*
2. *Цель*
3. *Задачи*
4. *Описание хода работы*
5. *Результат деятельности (скриншот **Диаграммы классов со стереотипами и** с помощью пиктограмм стереотипов, Диаграмма пакетов)*
6. **Ответьте на контрольные вопросы**
   - ***6.1. Сколько диаграмм классов строится для системы?***
   - ***6.2. Чем характеризуется объект?***
   - ***6.3. Что означает Индивидуальность объекта?***
   - ***6.4. Что такое стереотип?***
   - ***6.5. Управляющий класс определяет....***
   - ***6.6. Перечислите Граничные классы***
   - *6.7.* ***Назначение Пакета***